

Adnet MVP - правки и рекомендации к бизнес-плану

Дата: 19 июля 2026

Основа: `Adnet\_MVP\_business\_plan.pdf`

1. Общий вывод

Бизнес-модель выглядит реализуемой и технически здоровой: это не абстрактная биржа фриланса, а marketplace рекламных размещений и UGC-заказов с фиксированным процессом сделки. Сильная сторона документа - фокус на Telegram-first, безопасной сделке, проверке результата и ручной модерации на старте.

Главная правка: в документе нужно ещё жёстче отделить MVP от будущей платформы. Сейчас часть функций описана как желательная, но не всегда понятно, что именно строится в первой версии, а что относится к v2/v3.

2. Что стоит усилить в бизнес-модели

2.1. Чётче сформулировать первую продаваемую услугу

Сейчас продукт описан широко: Telegram-размещения, YouTube Shorts, VK, UGC, публикации, файлы, агентские кампании. Для MVP лучше зафиксировать первый коммерческий пакет.

Рекомендуемая формулировка:

Первая услуга Adnet: запуск микро-кампании из UGC-контента и/или размещений у проверенных Telegram/VK/YouTube-авторов по единому брифу, с резервом бюджета, проверкой результата и итоговым отчётом.

Это упростит продажи, разработку, модерацию и проверку результата.

2.2. Разделить два типа сделок

В документе UGC и размещения часто идут вместе, но технически это разные процессы.

Нужно явно описать два базовых типа сделки:

1. **Размещение**

Исполнитель публикует материал на своей площадке. Проверяются ссылка, срок размещения, обязательные элементы, публичные метрики.

2. **UGC-производство**

Исполнитель создаёт файл для рекламодателя без обязательной публикации. Проверяются файл, формат, длительность, ТЗ, права использования, версии правок.

Для MVP это можно вести в одной модели `Deal`, но с разными чек-листами проверки.

2.3. Добавить unit economics по маленьким сделкам

Комиссия 15% выглядит логично, но для маленьких чеков может не покрывать поддержку, модерацию и платежные расходы.

Нужно добавить правило:

- минимальная комиссия платформы;

- минимальный бюджет сделки;
- отдельная комиссия за ручное сопровождение или спор;
- ограничение количества бесплатных правок.

Пример:

Минимальная сумма сделки: 3 000-5 000 □. Минимальная комиссия платформы: 500-700 □. Ручное ведение пакета кампаний тарифицируется отдельно.

3. Технические правки к архитектуре

3.1. Сделать `Deal` центральной сущностью

Сейчас в документе есть кампании, заявки, платежи и проверки, но нужно сильнее подчеркнуть, что ядро системы - это не кампания, а сделка.

Рекомендуемая модель:

Campaign -> Application -> Deal -> Submission -> Review/Dispute -> Payment

`Deal` должен хранить:

- выбранного исполнителя;
- согласованную цену;
- комиссию платформы;
- дедлайны;
- тип услуги;
- права использования;
- статус;
- контрольный период;
- ссылку на платежную транзакцию;
- историю изменений.

3.2. Ввести state machine сделки

Статусы нужно описать как конечный автомат, а не просто список.

Рекомендуемые статусы:

```
application_sent
approved_waiting_payment
funded
in_progress
draft_submitted
revision_requested
ready_for_publication
submitted_for_review
under_control
completed
payout_sent
dispute_opened
cancelled
refunded
```

Для каждого статуса нужно указать:

- кто должен действовать дальше;
- какие действия разрешены;
- какие уведомления отправляются;
- какие таймеры запускаются;
- можно ли отменить сделку;
- что происходит с деньгами.

3.3. Добавить `AuditLog` как обязательный модуль

Для споров и выплат критично хранить все действия.

Нужно явно добавить:

- кто изменил статус;
- когда загружен файл;
- когда принята заявка;
- когда отправлен webhook от платежного провайдера;
- какие доказательства были приложены;
- кто из админов принял решение по спору.

Без `AuditLog` спорные сделки будет трудно разбирать.

3.4. Отдельно описать модуль доказательств

В документе доказательства упоминаются, но их стоит выделить как отдельную сущность.

Рекомендуемая сущность:

```
Evidence
- id
- deal_id
- submitted_by
- type: file | screenshot | publication_url | metric_snapshot | comment | admin_note
- url
- metadata
- created_at
```

Это позволит одинаково обрабатывать UGC-файлы, ссылки, скриншоты, комментарии и автоматические снимки метрик.

4. Проверка результата

4.1. Зафиксировать, что MVP не обещает performance

В документе это уже есть, но стоит повторить в разделе продукта и продаж:

В MVP Adnet проверяет факт выполнения условий сделки, но не гарантирует продажи, установки, лиды, охват или отсутствие накрутки.

Это важно юридически и коммерчески.

4.2. Сделать чек-листы по типам услуг

Для каждой услуги нужен отдельный чек-лист.

Пример для Telegram-размещения:

- пост существует;
- ссылка открывается;
- канал соответствует заявленному;
- рекламная ссылка/промокод присутствует;
- маркировка рекламы есть, если требуется;
- пост не удалён в течение контрольного срока;
- базовые публичные метрики сохранены.

Пример для UGC:

- файл загружен;
- формат соответствует ТЗ;
- длительность соответствует ТЗ;
- обязательные сцены/тезисы присутствуют;
- права использования подтверждены;
- количество правок не превышено.

5. Платежи и безопасная сделка

5.1. Не использовать внутренний баланс в MVP

Это правильная мысль в документе, её нужно сделать архитектурным принципом.

Рекомендуемая формулировка:

Adnet не хранит деньги пользователей на внутреннем балансе. Платформа хранит только идентификаторы платежей, статусы инвойсов, резервов, возвратов и выплат.

5.2. Заранее выбрать платежную модель

Нужно добавить раздел с вариантами:

- marketplace-платёжный провайдер;
- агентская схема;
- номинальный счёт;
- ручной юридически согласованный пилот.

Для каждого варианта нужно указать:

- кто принимает деньги;
- кто платит исполнителю;
- кто выдаёт документы;
- как делается возврат;
- как удерживается комиссия.

6. MVP-скоуп, который стоит зафиксировать

Рекомендуемый MVP должен включать:

- Telegram auth;

- две роли пользователя;
- профиль рекламодателя;
- профиль исполнителя;
- подключение площадок вручную или полуавтоматически;
- создание кампании;
- модерацию кампании;
- ленту заданий;
- заявки;
- одобрение заявки;
- сделку со статусами;
- загрузку UGC-файлов и ссылок;
- доказательства выполнения;
- ручную проверку;
- спор;
- платежные статусы;
- админ-панель;
- уведомления в Telegram;
- итоговый отчёт по сделке.

Не включать в первый MVP:

- гарантии охвата и продаж;
- автоматическое распределение бюджета;
- сложный performance-трекинг;
- полноценный агентский кабинет;
- TikTok/Instagram как обязательные площадки;
- внутренний P2P-баланс;
- мобильное приложение.

7. Рекомендуемый технический стек

Для быстрого MVP:

- **Bot:** TypeScript + grammY или Telegraf.
- **Mini App:** Next.js / React.
- **Backend:** NestJS или FastAPI.
- **Database:** PostgreSQL.
- **Queue:** Redis + BullMQ / Celery.
- **Storage:** S3-compatible storage.
- **Admin:** Next.js admin или готовый admin framework.
- **Payments:** внешний провайдер с webhooks.
- **Monitoring:** Sentry + структурированные логи.

Если делать быстро и цельно, я бы выбрал:

TypeScript stack:
Telegram bot + Next.js Mini App + NestJS API + PostgreSQL + Redis + S3

Причина: единый язык, хорошая экосистема Telegram, удобная разработка Mini App и API.

8. Главные риски проекта

Технические

- нестабильные или ограниченные API площадок;
- сложность автоматической проверки видео/публикаций;
- большое количество edge cases в спорах;
- платежные webhooks и возвраты;
- хранение файлов и доказательств;
- необходимость точного audit trail.

Операционные

- ручная модерация может стать узким местом;
- маленькие сделки могут быть нерентабельны;
- исполнители могут срывать сроки;
- рекламодатели могут спорить по субъективному качеству;
- сложные категории рекламы требуют комплаенса.

Продуктовые

- риск стать обычной фриланс-биржей;
- слишком широкий старт по площадкам;
- недостаточная ликвидность: мало заявок на кампанию;
- слабая повторяемость рекламодателей.

9. Что добавить в документ перед разработкой

1. Таблицу MVP / Later / Never.
2. State machine сделки.
3. Чек-листы проверки по каждому типу услуги.
4. Платежную схему с ролями сторон.
5. Список экранов MVP.
6. API-события и webhooks.
7. Политику отмены и спора.
8. Минимальную комиссию и минимальный чек.
9. Роли админки.
10. Метрики операционной нагрузки: ручное время на сделку, доля споров, доля автопроверок.

10. Итоговая рекомендация

Проект стоит реализовывать как concierge marketplace с постепенной автоматизацией. На старте важно не пытаться автоматизировать всё. Лучше провести первые кампании почти вручную, но уже внутри правильной технической модели: пользователи, кампании, заявки, сделки, доказательства, статусы, платежи и аудит.

Технически MVP реалистичен. Основная сложность - не в разработке интерфейса, а в правильной сделочной логике, платежной модели, проверках и админских процессах.